君正®

Zeratul T23 快起效果调试说明

Date: Jun.2023

北京君正集成电路股份有限公司
Ingenic Semiconductor Co., Ltd.

君正®
Zeratul T23 快起效果调试说明
Copyright © Ingenic Semiconductor Co. Ltd 2019. All rights reserved.

Release history

Date	Revision	Change
Jun.2023	1.0	First release

Disclaimer

This documentation is provided for use with Ingenic products. No license to Ingenic property rights is granted. Ingenic assumes no liability, provides no warranty either expressed or implied relating to the usage, or intellectual property right infringement except as provided for by Ingenic Terms and Conditions of Sale.

Ingenic products are not designed for and should not be used in any medical or life sustaining or supporting equipment.

All information in this document should be treated as preliminary. Ingenic may make changes to this document without notice. Anyone relying on this documentation should contact Ingenic for the current documentation and errata.

北京君正集成电路股份有限公司

地址: 北京市海淀区西北旺东路 10 号院东区 14 号楼君正大厦
电话:(86-10)56345000
传真:(86-10)56345001
Http: //www.ingenic.com

目录

1 起始 EV 1
1.1 什么叫做起始 EV 1
1.2 T23 RISCV 1
1.3 起始 EV 参数代码位置 1
1.4 起始 EV 参数表格如何创建 2
2 测试环境 4
2.1 白天模式 4
2.2 夜视环境 4
2.3 没有均匀光源 5
3 IRCUT、IRLED 处理方法 6
3.1 Bootloader 中的处理方法 6
3.2 内核中的处理方法 7

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

1 起始 EV

1.1 什么叫做起始 EV

普通 IPC 在开机使用的时候，画面有一个由暗变亮的过程，这个过程称之为 AE 收敛过程。电池 IPC 由于每次启动都是冷启动，如果不做特殊处理，必然也会存在这样一个收敛过程，这个对于用户体检极差。

为了缩短这个过程，我们提供个一个方法，在开机的时候，可以设置一个起始的曝光参数，收敛就从设置的起始曝光参数开始收敛，这样就会缩短 AE 收敛过程。设置的这个曝光参数，我们称之为起始 EV 参数。

1.2 T23 RISCV

T23 平台包含 RISCV 小核，可以提供一些辅助处理，当前我们把 ISP 的 AE、AWB 功能移植到了 RISCV 上面，由于我们的系统是 linux 系统，从 BootLoader 到 linux 内核启动需要一定的时间，在 RISCV 的加持下，可以并行初始化 ISP、Sensor，并对 AE、AWB 进行收敛，内核期间 ISP 驱动注册的时候，图像效果可以从 RISCV 斜街过来，这样内核拿到的图像数据基本是稳定可用的。

1.3 起始 EV 参数代码位置

起始 EV 参数存放在 TAG 分区中，在 BootLoader 阶段，通过 ADC 得到光敏信息并且与 TAG 中的 EV 参数进行匹配，得到起始 EV 值，这个 EV 值会被 RISCV 使用，作为 RISCV 的起点。

起始 AE 信息存放在 tools/make_tag/sensor_start_ae_table 目录下

libz_sensor.c 为白天 ADC 和 EV 对应关系表。

struct vol_start_value_table table_day[] 为白天模式对应关系表

struct vol_start_value_table table_night[] 为夜视模式下对应关系表

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

1.4 起始 EV 参数表格如何创建

我们需要在 BootLoader 中获取 ADC 值来匹配 EV 值，由于从上电到 BootLoader 启动速度很快，部分光敏其实在这个时候还不是很稳定，所以我们按照之前 T20、T30 的方式在系统起来后，通过工具来获得 ADC 和 EV 对应关系，其实与实际使用情况是不准确的，为了达到更准确的效果，我们再建立表格的时候，就需要用 BootLoader 中读取的 ADC 值作为起始 AE 参数。与真实环境一致。

1、打开 BootLoader 打印信息：

如下，在 tools/make_tag/tag_generator/build_tag_t23.sh 文件中，对应的环境变量后面增加 debug 字符串，然后重新编译 tag.bin 即可。

再对应将 tag.bin 烧录到 tag 分区，启动过程中，bootloader 会打印对应的 ADC 值。

```
12
13 if [ ${ZRT_ENV_SOC_TYPE} = SOC_T31Z ]; then
14     CMDLINE="console=${UART},115200n8 mem=80M@0x0 rmem=48M@0x5000000 root=/dev/ram0 rw rdinit=/linuxrc mtdparts=jz_sfc:256K(boot),352K(tag)
   512k(config),16M@0(all) lpj=6955008 ${QUIET}"
15     BOOTINFO="kernel=5M@0x98000 rootfs=3M@0x598000"
16     ENVINFO="senv;[HW];init_vw=1920;init_vh=1080;nrvbs=3;mode=0;eenv;debug;"
17     SOC_TYPE=t31
18     RISCV_FW="../riscv_fw/riscv_fw_${ZRT_ENV_SENSOR}.bin"
19 fi
20
21 if [ ${ZRT_ENV_SOC_TYPE} = SOC_T31A ]; then
22     CMDLINE="console=${UART},115200n8 mem=80M@0x0 rmem=48M@0x5000000 root=/dev/ram0 rw rdinit=/linuxrc mtdparts=jz_sfc:256K(boot),352K(tag)
   512k(config),16M@0(all) lpj=6955008 ${QUIET}"
23     BOOTINFO="kernel=5M@0x98000 rootfs=3M@0x598000"
24     ENVINFO="senv;[HW];init_vw=1920;init_vh=1080;nrvbs=3;mode=0;eenv;"
25     SOC_TYPE=t31
26     RISCV_FW="../riscv_fw/riscv_fw_${ZRT_ENV_SENSOR}.bin"
27 fi
28
29 SENSOR_SETTING_MODE0="../sensor_calibration_bin/${ZRT_ENV_SENSOR}_${SOC_TYPE}.bin"
30 SENSOR_AE_TABLE="../sensor_start_ae_table/ae_table_${ZRT_ENV_SENSOR}.bin"
31
32 cd ${ZRT_ENV_TOOLS_DIR}/make_tag/sensor_start_ae_table/
33 make clean && make
34
35 cd ${ZRT_ENV_TOOLS_DIR}/make_tag/sensor_init_setting/
36 make ${ZRT_ENV_SENSOR}_${SOC_TYPE}_init
37 cp ${ZRT_ENV_SENSOR}_${SOC_TYPE}_init.bin ${ZRT_ENV_TOOLS_DIR}/make_tag/tag_generator/
38
39 cd ${ZRT_ENV_TOOLS_DIR}/make_tag/tag_generator/
40 make distclean && make tag_generator_t31_host
41
42 if [ -f ${RISCV_FW} ]; then
/work/zeratul/release tools/make_tag/tag_generator/build_tag_t31.sh[+][POS=1,12][1%]27/09/20 - 19:49
```

如果不想编译 tag，也可用通过 tag_generator 来处理，在串口执行以下命令，在原有的环境变量后面，增加 debug 字串即可。

```
    tag_generator --update --tag_path=/dev/mtdblock1 --
env="senv;[HW];init_vw=1920;init_vh=1080;nrvbs=3;mode=0;eenv;debug;"
```

在 bootloader 打印信息如下：

```
    g_adc_value:%d, ADC:%d, EV:%d, is night:%d
```

2、打开内核打印信息：

源码位置：os/drivers/tx-isp-t23/tx-isp-fast.c，将以下截图处 debug 打开，重新编译后，并重新编译内核。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
    }

    printk("Riscv frame count:%d\n", frame_count);

    g_is_night = tx_isp_get_riscv_night_mode();
    printk("Tuning mode is:%d\n", g_is_night);

    //g_riscv_sensor_again = 4096;
    //g_riscv_sensor_inttime = 1242;

    // Set Sensor Attr (again、inttime)
    //tx_isp_set_ev_start(g_riscv_isp_ev);
    //tx_isp_set_sensor_again(g_riscv_sensor_again);
    //tx_isp_set_sensor_inttime(g_riscv_sensor_inttime);
    //tx_isp_set_isp_gain(4096);

    //tx_isp_discard_frame(0xfffffffc); /* 丢弃前两帧 */

    // enable debug info per frame
    tx_isp_enable_debug(1, 100);

    return 0;
}
```

烧录新编译的 tag 和 kernel 之后，启动，BootLoader 的信息会从串口打印出来，内核信息可以进入系统后通过执行 dmesg 打印出来，我们只需要将 BootLoader 中的 ADC 值，以及 dmesg 中稳定后的 EV 值保存到起始 EV 表格中即可。

dmesg 信息取最后一次稳定的值即可，如下：

```
[    2.379147] ISP [35] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    2.445384] ISP [36] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    2.512482] ISP [37] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    2.578711] ISP [38] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    2.645369] ISP [39] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    2.712033] ISP [40] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    2.778693] ISP [41] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    2.845360] ISP [42] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    2.912454] ISP [43] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    2.978680] ISP [44] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    3.045344] ISP [45] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    3.112585] ISP [46] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    3.179245] ISP [47] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    3.245904] ISP [48] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    3.312574] ISP [49] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
[    3.379236] ISP [50] intt:3276 ev:3781 expr_us:60000 again:5 dgain:0 gain_log2:0 total_gain:295
```

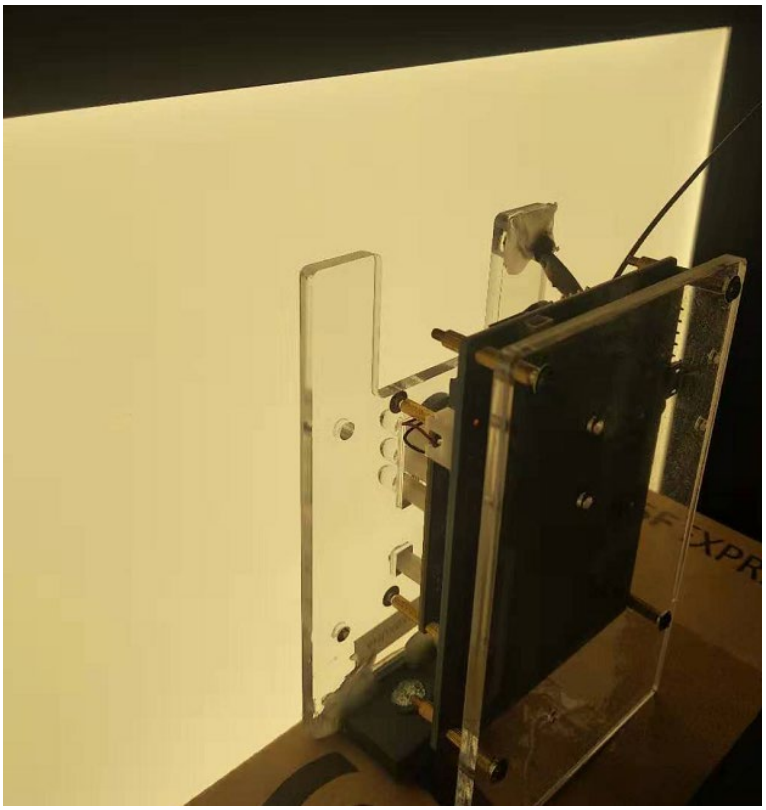

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

2 测试环境

2.1 白天模式

取一个均匀光源，放到暗室中，不要有外界光线干预，如下图放置设备，镜头可以尽量贴紧。

然后调节光源的亮度，可用照度计记录对应的照度值，从暗到亮，逐步调节，调节之后，待亮度稳定后，开关设备，通过串口读取 ADC 和 EV 值保存即可。



2.2 夜视环境

由于一般的光敏对红外光不敏感，也就是红外灯开或者关，光敏的 ADC 值基本不变，所以夜视情况针对不同的使用场景，效果会不一样。在夜视情况下，无法做到统一。所以一般我们会选择一个较为适中的情况进行测量。

在暗室中选择一个场景，将设备与目标参照物放置一定距离，如 5 米。然后将设备切到夜视模式，打开红外和 IRCUT，然后将均匀光源放置到某个角落，避免光线直射到光敏上，或者直射到镜头里面，造成采样不准。

在这种情况下，调节均匀光源光线强度，然后记录 ADC 和 EV 值即可。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

2.3 没有均匀光源

在没有均匀光源设备的情况，可以将设备放置到一个实际应用场景中，然后插上 USB 将设备常供电，防止电池电量耗尽。

写一个小的测试程序，10 秒左右获取一次 ADC 和 EV 的值，如果 ADC 相同的情况，可以将 EV 值进行取平均即可。

在设备上跑这个程序，然后在实际场景中跑上 1 到 2 天即可。

ADC 读取 API 位于头文件中：
userland/5.4.0/imp-t23/include_cn/sysutils/su_adc.h

通过 SU_ADC_GetChnValue 函数获取 ADC 值

```
/**
 * @fn int SU_ADC_GetChnValue(uint32_t chn_num, int *value);
 *
 * 得到第chn_num通道的ADC值
 *
 * @param[in] chn_num 第几路通道
 *
 * @param[out] value 得到的ADC值
 *
 * @retval 0 成功
 * @retval 非0 失败，返回错误码
 *
 * @remark 无
 *
 * @attention 无
 */
int SU_ADC_GetChnValue(uint32_t chn_num, int *value);
```

对应的 EV 值，可用通过以下接口获得，如下：

```
/**
 * ISP EV 参数。
 */
typedef struct {
    uint32_t ev;            /**< 曝光值 */
    uint32_t expr_us;       /**< 曝光时间 */
    uint32_t ev_log2;       /**<log格式曝光时间 */
    uint32_t again;         /**< 模拟增益 */
    uint32_t dgain;         /**< 数字增益 */
    uint32_t gain_log2;     /**< log格式增益 */
}IMPISPEVAttr;

/**
* @fn int IMP_ISP_Tuning_GetEVAttr(IMPISPEVAttr *attr)
*
* 获取EV属性。
* @param[out] attr EV属性参数
*
* @retval 0 成功
* @retval 非0 失败，返回错误码
*
* @attention 在使用这个函数之前，IMP_ISP_EnableTuning已被调用。
*/
int IMP_ISP_Tuning_GetEVAttr(IMPISPEVAttr *attr);
```

备注：

选择的场景尽量能包含全黑到较亮范围。

夜视情况下，程序需要将 ISP 切换到夜视模式，IRCUT、IRLED 需要对应处理。这样制作出来的表格才准确。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

3 IRCUT、IRLED 处理方法

IRCUT 切换需要一定的时间，从 BootLoader 到内核期间，是有足够的时间来切换 IRCUT，我们需要保证内核得到的第一帧数据是已经切换完成并正常可用的即可。

3.1 Bootloader 中的处理方法

在 BootLoader 阶段，我们读取 ADC 之后，通过阈值判断后，直接打开或关闭 LED、IRCUT。

IRCUT、IRLED 等设置在 tools/make_tag/sensor_start_ae_table/ 对应的 Sensor 文件中。如下图：

```
#include "libz_common.h"

int g_gpio_led_mode = LED_MODE_HIGH_ON;
int g_gpio_led = GPIO_PB(30);
int g_gpio_ircut_mode = IRCUT_MODE_LEVEL_TRIGGER_HIGH;
int g_gpio_ircut_p = GPIO_PB(18);
int g_gpio_ircut_n = GPIO_PB(28);
int g_gpio_ircut_edge = 0;
int g_adc_value = 100;
int g_adc_direction = ADC_DIRECTION_0;
```

具体参数含义如下图，对应的宏定义请参考 libz_common.h 文件说明

```
/**
 * IRLED
 * -1: 不受控制
 * 0: 高电平亮灯, 低电平灭灯
 * 1: 高电平灭灯, 低电平亮灯
 */
int gpio_led_mode;
int gpio_led;

/**
 * IRCUT
 * -1: 不受控制
 * 0: 电平触发方式 P 高电平 N 低电平 切夜视
 * 1: 电平触发方式 N 高电平 P 低电平 切夜视
 * 2: 沿触发方式 上升沿触发 切夜视
 * 3: 沿触发方式 下降沿触发 切夜视
 */
int gpio_ircut_mode;
int gpio_ircut_p;
int gpio_ircut_n;
int gpio_ircut_edge; /* 当触发方式为沿触发方式，当前成员参数GPIO值生效 */

// ADC 阈值
int adc_value; /* 白天夜视切换阈值,该值小于0,不受控制 */
int adc_direction; /* 0: 小于阈值为夜视 1:大于阈值为夜视 */
```

如果不想让 uboot 控制任何数据，对应的参数都设置为-1 即可，如下：

```
#include "libz_common.h"

int g_gpio_led_mode = LED_MODE_IGNORE;
int g_gpio_led = -1;
int g_gpio_ircut_mode = IRCUT_MODE_IGNORE;
int g_gpio_ircut_p = -1;
int g_gpio_ircut_n = -1;
int g_gpio_ircut_edge = 0;
int g_adc_value = -1;
int g_adc_direction = ADC_DIRECTION_0;
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

3.2 内核中的处理方法

内核处理代码位于 os/drivers/tx-isp-t23/tx-isp-fast.c，内核期间主要处理两件事情，一是防止 IRCUT 耗电以及烧毁，需要在内核期间恢复到 idle 状态，二是将创建 ir_status 节点，供上层应用获取内核处理状态。防止频繁切换。

```
// IRCUT 切回idle状态
ircut_gpio_init();
// 创建 ir_status 节点
proc_create("ir_status", S_IRUGO, NULL, &ir_status_proc_fops);
```

提供一个节点供上层读取相关信息 /proc/ir_status

该节点有 5 种状态，读取的值为字符串，分别为：

ignore：未设置光敏范围和红外灯以及偏光片回调时，不处理，返回 ignore
off: 用戶层设置 IR mode 为常关（白天状态）
on: 用戶层设置 IR mode 为常开（夜晚状态）
day: 用戶层设置 IR mode 为自动切换，kernel 判断光敏为白天状态
night: 用戶层设置 IR mode 为自动切换，kernel 判断光敏为夜晚状态

注意：IRCUT 由于是电磁装置，在 BootLoader 开启，如果不归位，电磁线圈会一直通电，导致功耗损失，长时间后，会导致 IRCUT 烧坏，所以需要在内核阶段切回 idle 状态。

另外，由于黑白夜视在 BootLoader 中就需要判断，为了内核能够很好的衔接，需要将白天夜视状态传递到内核。实际的传递方向为 BootLoader ---> RISCV ----> Kernel.

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.